arXiv:2604.09860v4 [cs.RO] 14 Aug 2026

RoboLab: A High-Fidelity Simulation Benchmark for Analysis of Task Generalist Policies

Jenai Xuning Yang[1], Rishit Dagli[2,4], Alex Zook[1], Hugo Hadfield[1],
Ankit Goyal[1], Stan Birchfield[1], Fabio Ramos[1,3], and Jonathan Tremblay[1]
[1]NVIDIA, [2]University of Toronto, [3]The University of Sydney, [4]Work done during internship at NVIDIA

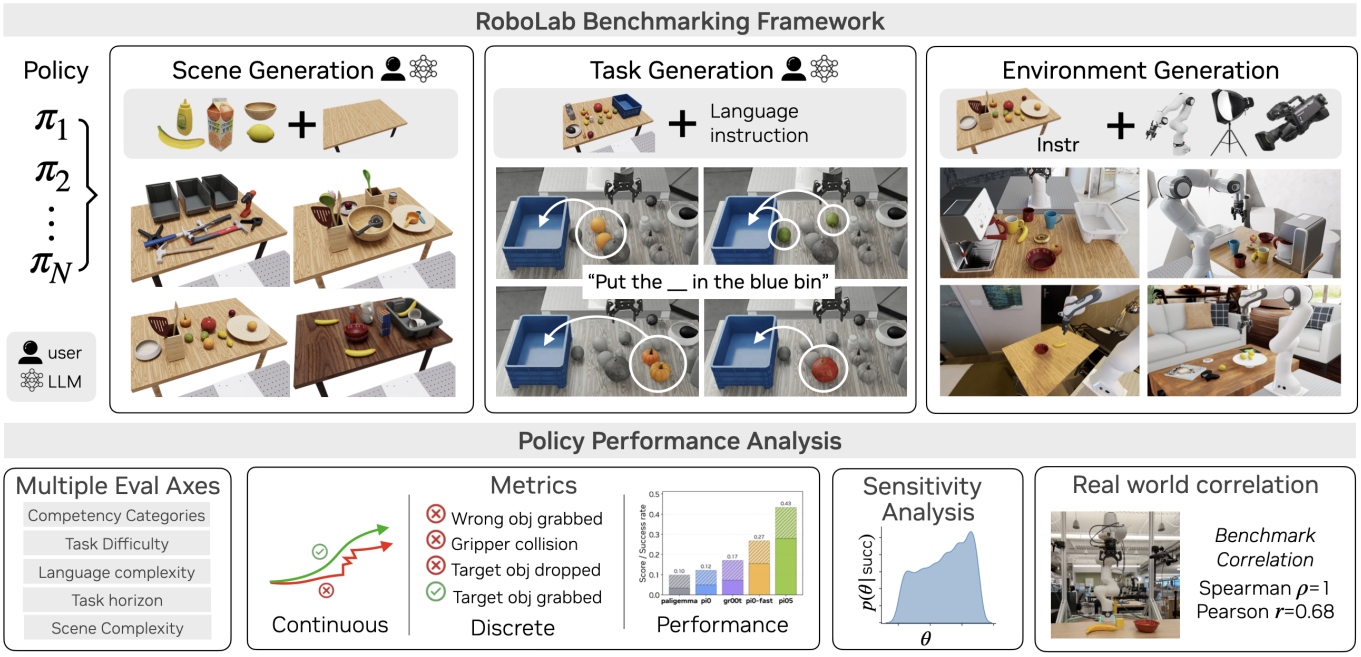


Fig. 1: **Overview of RoboLab.** RoboLab addresses large-scale evaluation of real-world policies via a novel evaluation framework. By introducing a streamlined pipeline for generating new scenes and tasks (top row), RoboLab enables rapid extensibility for testing generalization capabilities. The accompanying benchmark, *RoboLab-120*, introduces various evaluation axes and a suite of metrics and analysis tools, and demonstrate benchmark-level correlation with real-world benchmarks (bottom row).

***Abstract*—The pursuit of general-purpose robotics has yielded impressive foundation models, yet simulation-based benchmarking remains a bottleneck due to rapid performance saturation and a lack of true generalization testing. Existing benchmarks often exhibit significant domain overlap between training and evaluation, trivializing success rates and obscuring insights into robustness. We introduce RoboLab, a simulation benchmarking framework designed to address these challenges. Concretely, our framework is designed to answer two questions: (1) to what extent can we understand the performance of a real-world policy by analyzing its behavior in simulation, and (2) which factor most strongly affect policy behavior. First, RoboLab enables human-authored and LLM-enabled generation of scenes and tasks in a robot- and policy-agnostic manner within a high-fidelity simulation environment. We introduce an accompanying RoboLab-120 benchmark, consisting of 120 tasks categorized into three competency axes: visual, procedural, relational, across three difficulty levels. Second, we introduce a systematic analysis of real-world policies that quantify both their performance and the sensitivity of their behavior to controlled perturbations, exposing significant performance gap in current state-of-the-art models. By providing granular metrics and a scalable toolset, RoboLab offers a scalable framework for evaluating the true generalization capabilities of task-generalist robotic policies. Project website: https://research.nvidia.com/labs/srl/projects/robolab/.**

I. Introduction

The pursuit of generality has been a longstanding challenge in modern robotics. Recent advances have produced impressive generalist robot policies that demonstrate success in challenging and novel tasks in the real-world. Despite this progress, benchmarks for evaluating whether these policies are truly task-general has been slow. Evaluating models in the real world remains prohibitively expensive and logistically expensive, motivating the rise of simulation-based benchmarks as an appealing alternative.

Current robotics benchmarks [18, 36, 10, 15] face several critical limitations: (1) a lack of high-fidelity simulation aimed at evaluation of real-world policies; (2) rapid performance saturation on static task sets; and (3) a lack of granular analysis regarding policy failure modes.

For instance, popular benchmarks like LIBERO [18] often utilize nearly identical environments for both training and

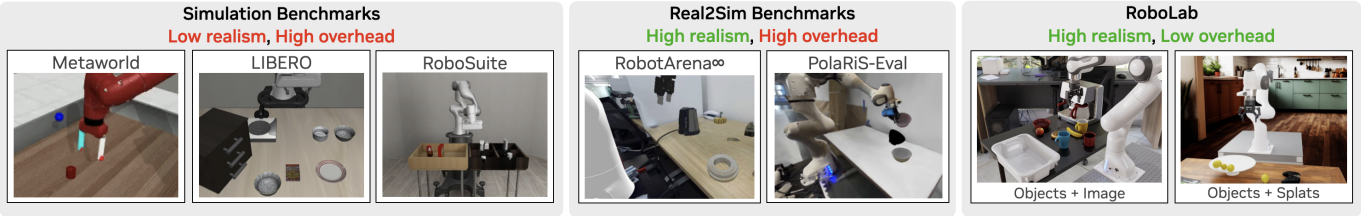


Fig. 2: Three approaches for robotic benchmarks. LEFT: To date, pure simulation based benchmarks have exhibited low visual quality, creating a large sim2real transfer gap. MIDDLE: Real2sim benchmarks address this issue by using techniques to bring real-world visual texture into simulation. However, these environments are extremely costly with reported per-scene generation time of ∼1hr [9]. RIGHT: Our approach achieves a high degree of realism with low overhead.

evaluation. When policies are fine-tuned on these simulation-specific demonstrations, the lack of a meaningful task domain gap trivializes the evaluation process and obscures the model's true generalization capabilities. Many existing platforms have limited realism or are difficult to extend due to using rigid architectures that make it cumbersome to introduce new objects, tasks, or robots (Fig. 2).

To address these limitations, we present RoboLab (Fig. 1), a benchmarking framework designed for rigorous evaluation of generalist policies trained on real-world data. Unlike prior benchmarks that rely on PDDL or rigid scene-graph definitions [18], RoboLab introduces an easy-to-use framework that enables human-authored and AI-enabled scene and task generation. This enables fast creation of hundreds of new tasks and scenes, providing a scalable framework that mitigates benchmark saturation and ensures long-term value.

We introduce the RoboLab-120 benchmark, comprising of 120 hand-curated diverse pick-and-place tasks, aimed at evaluating the generalization capabilities. These tasks span varying difficulties (65 simple, 38 moderate, 18 complex) and multiple competency axes (44 relational, 91 visual, and 36 procedural). This benchmark reflects tasks encountered in "in the wild" household scenarios. To prevent evaluation of policies overfitted to a particular simulation domain, we evaluate policies trained exclusively on the real-world DROID [12] dataset. Our analysis shows that the results on RoboLab-120 achieve benchmark-level correlation with real-world benchmarks [28], indicating that RoboLab is a meaningful proxy for real-world evaluation for task-generalism.

Lastly, we introduce a suite of analysis tools that gives insight into the model performance beyond binary success rates and broader understanding of policy performance; including subtask scoring, event tracking, Bayesian-based sensivity analysis of performance given scene variations via Neural Posterior Estimation.

In summary, our contributions are:

1) **RoboLab**: A novel benchmarking platform designed for evaluating real-world robotics policies with a scalable, AI-based workflow capable of procedurally generating over hundreds of unique robot- and policy- agnostic scenes and tasks, built on IsaacLab[8].
2) **RoboLab-120 Benchmark**: 120 tasks diversely represented across three distinct competency axes (visual, procedural, relational), across 3 levels of difficulty, and supported by robustness metrics.
3) **Policy Analysis**: We introduce a suite of analysis tools that gives insight into the model performance beyond binary success rates and broader understanding of policy performance.

II. Related Work

Simulation-Based Benchmarks. Simulation provides a scalable and reproducible environment for evaluating robot manipulation policies. Widely used benchmarks such as RLBench [10], MetaWorld [34], and robosuite [37], ManiSkill2 [6], CALVIN [19], LIBERO [18], and BEHAVIOR-1K [14], offer standardized task suites for learning and evaluation in simulation across pre-defined task families and object configurations. However, in these settings, policies are typically trained and evaluated in the same simulated environments, which encourages overfitting to simulator-specific quirks, leads to rapid benchmark saturation, and makes real-world generalization hard to assess. [36]. In our setting, policies are instead trained on large-scale real-world data (e.g., DROID [12]), while high-fidelity simulation is used only as a controlled evaluation environment. Training and evaluation domains are decoupled and measured performance more closely reflects robustness in the real world. Some benchmarks also solely focus on perturbations and variations to probe policy robustness (LIBERO [18], REALM [29]); but on a limited task set. RoboLab's task generation and diagnostic analysis allows large-scale evaluation and performance analysis, complementary to existing benchmarks.

Real-to-sim Evaluation. Recent work have focused on leveraging 3D reconstruction to build photorealistic simulation scenes from real-world videos in order to achieve closer visual alignment between simulation and real world photorealism [15, 11, 9, 38]. These works typically use Gaussian splatting, 3D segmentation, and multi-view inpainting, often operated at a per-scene level, which entails costly optimization and makes it slow to scale beyond a small number of environments [9, 35, 27] (Appendix A-B). In contrast, our framework produces large-scale, photorealistic scenes and tasks within minutes rather than hours, while preserving sufficient geometric and visual fidelity for policy evaluation, thereby making real-

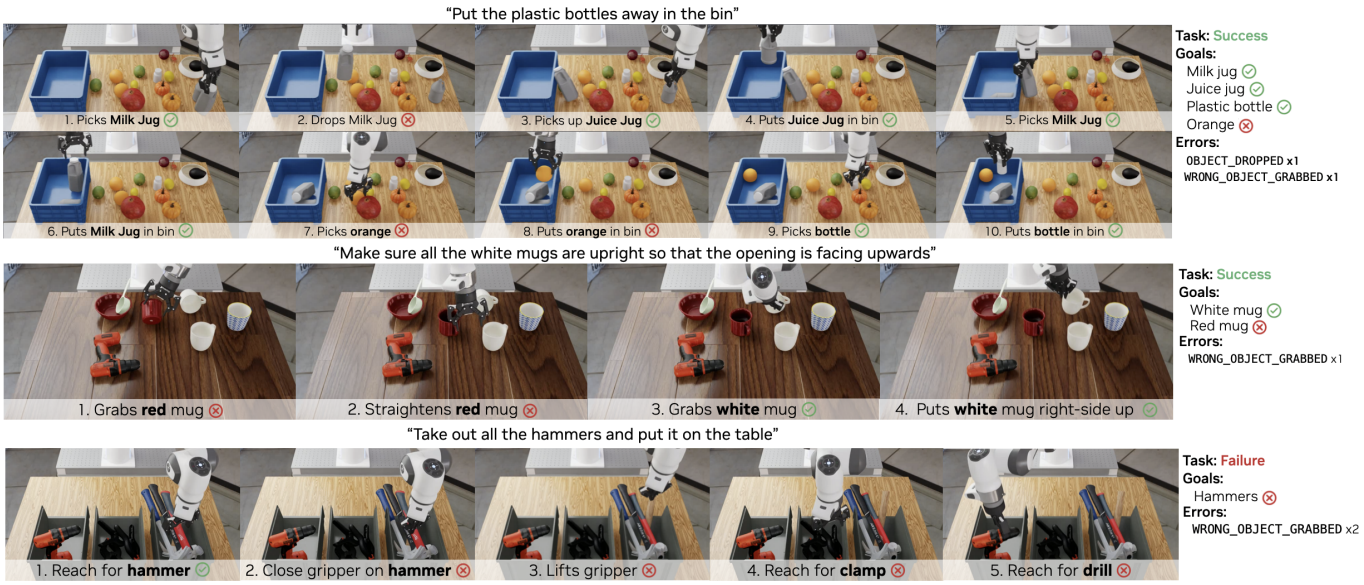


Fig. 3: **Task progression of a few tasks, illustrating errors encountered during policy rollout.** Top row: Although the task is successfully completed, errors were encountered during execution: 1) The robot drops the milk jug too early, missing the bin. 2) the robot grasps an orange (wrong object) and puts it in the bin. Mid row: An extraneous object was reoriented before the actual intended object. Final row: Intended objects were attempted unsuccessfully, and the policy tended to two wrong other objects.

to-sim benchmarking practical at the scale needed for modern generalist robot policies.

III. RoboLab

Evaluating real-world, generalist robotics policies in simulation remains a significant challenge. RoboLab aims to address this with a few design principles: 1) Enable easy and fast scene/task generation; 2) Tasks in RoboLab are policy- and robot- agnostic, enabling comparison of multiple policy and robot choices when it comes to solving real-world tasks; and 3) generate tasks that are diverse, representative of real-world tasks, and enable a multifaceted analysis of models to understand their generalization capabilities.

A. RoboLab Scene and Task Generation

RoboLab introduces a a user-friendly workflow that mirrors the process of preparing a real-world robot evaluation (Fig. 1): 1) First, create a **scene** by positioning and orienting objects in a workspace; 2) Then, define a **task** as language instructions for a goal state in the scene; 3) Lastly, instantiate an **environment** by selecting a robot, policy, and variations of scene features including camera, lighting, and backgrounds for a task. RoboLab enables evaluation of the same tasks across different robot embodiments. By deferring robot- and experiment-specific binding to runtime, we facilitate systematic evaluation of tasks across robot configurations and policy variants, without having embodiment-specific training.

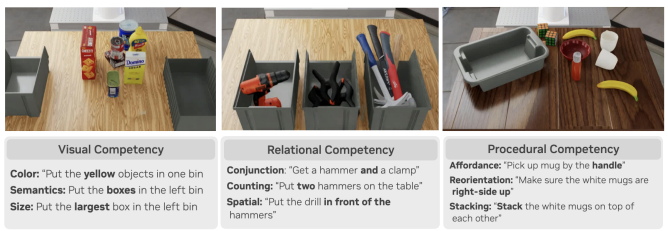


Fig. 4: Example of language instructions in RoboLab-120.

Formally, define a *scene* $S = \{(b_i, \mathbf{p}_i, \mathbf{q}_i)\}_{i=1}^{N}$, where b_i represents an object instance selected from the available catalog of objects $\mathcal{B}$ and $\mathbf{p}_i \in \mathbb{R}^3, \mathbf{q}_i \in SO(3)$ denote its position and orientation. Define a *task* $\mathcal{T} = \{S, l\}$, where l is the *language instruction* to complete in the scene. Define a policy $\pi : \mathcal{O} \to \mathcal{A}$ where the action space $\mathcal{A} \in \{\mathcal{A}^{\text{joint}}, \mathcal{A}^{\text{EE}}, \dots\}$ and observation space $\mathcal{O} = (\mathcal{O}^{\text{proprio}}, \mathcal{O}^{\text{rgb}}, \mathcal{O}^{\text{depth}} \dots)$ is policy dependent. An environment $E = (\mathcal{T}, \mathcal{R}, \mathcal{O}, \mathcal{A}, \xi)$ consists of a task, robot embodiment $\mathcal{R}$, policy parameters $(\mathcal{A}, \mathcal{O})$, and scene variations $\xi = (\xi^{\text{camera}}, \xi^{\text{light}}, \xi^{\text{background}}, \xi^{\text{pose}})$. More details on the specific objects, scenes and tasks in RoboLab can be found in Appendix A. To scale scene and task generation, we introduce an automated workflow, described below:

1) Scaling Scene Generation: We enable scaling scene generation through an automated pipeline that: 1) prompts an LLM to generate a structured scene plan for asset placement; 2) uses a geometric solver and physics simulation to check asset placement validity; and 3) refines the scene if it is not valid. First, the LLM is prompted with a theme (e.g., "messy counter") to generate a structured scene plan consisting of a subset of objects $B \subset \mathcal{B}$ and spatial predicates $\mathcal{P}$ governing the layout. The LLM is provided with the full catalog of objects $\mathcal{B}$ containing names and bounding box dimensions $\mathbf{d}_i \in \mathbb{R}^3$. Second, a spatial solver converts the relational predicates $\mathcal{P}$ into valid pose configurations $(\mathbf{p}, \mathbf{q})$). Objects are processed in dependency order, with support surfaces placed before objects on those surfaces (Algorithm 1). To check physical stability,

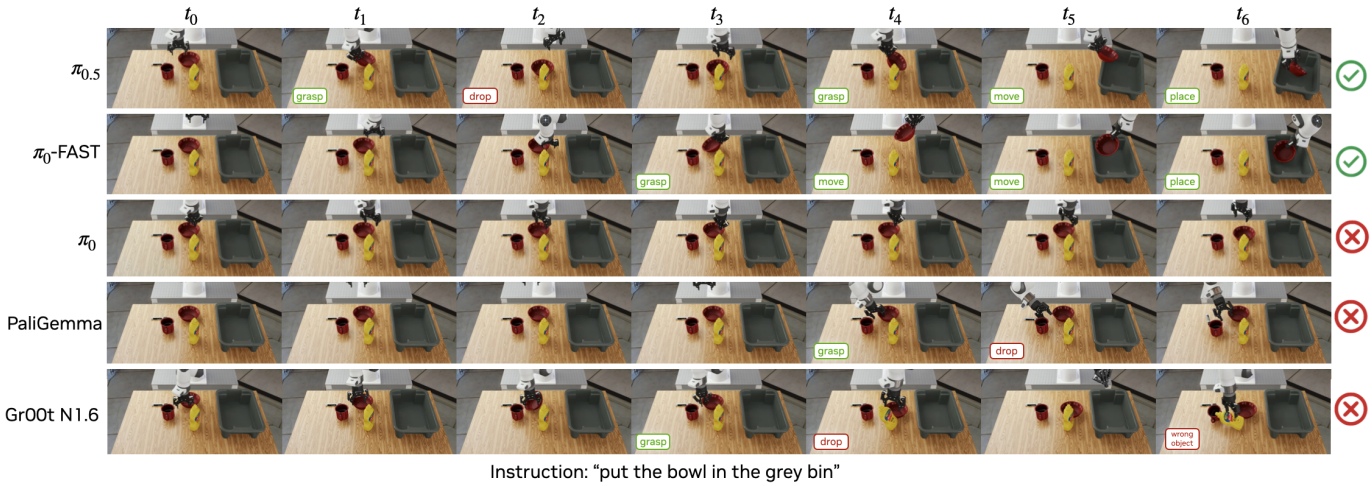


Fig. 5: Comparison of policy performance for bowl-in-bin manipulation. Rows represent distinct policies shown in chronological order (left to right). Successful execution involves grasping the central red bowl and depositing it into the gray bin on the right. Unsuccessful attempts are characterized by aimless arm trajectories and a lack of object interaction.

the scene is then forward simulated in Isaac Sim [21] for 300 steps under gravity. An object b_i is flagged as *unstable* if it's maximum Euclidean displacement is larger than a threshold (typically 0.02m). Third, If any object is unstable, we generate a text error describing the failure (e.g., "Object 'apple' fell off 'plate' with displacement 0.15m"). This feedback is provided to the LLM to refine the scene plan and repeat the process. Further details, including on the spatial and physical solvers, are provided in Appendix C.

2) Scaling Task Generation: We enable scaling task generation through an automated pipeline that: 1) generates task code from information including the scene and competency axes; 2) validates code syntax; 3) validates asset selections in the scene; and 4) refines the task if it is not valid. First, we prompt an LLM with detailed task information: 1) the scene object catalog B_S and metadata with dimensions; 2) task examples demonstrating the task structure; 3) the complete predicate library defining sub-task success and termination; 4) Competency-axes language templates with placeholders for objects, spatial verbs, and attributes; and 5) constraints including difficulty levels and physical feasibility requirements (e.g., containment size constraints, stacking stability). Second, tasks are check for syntax validity as code. Third, asset validation checks that all objects are not in the forbidden set and, for containment tasks (e.g., "place b_i inside b_j"), that inner objects fit inside containers with some clearance. Fourth, if validation fails, feedback is gathered into a fix prompt $\mathcal{Q}_{\text{fix}}$ that includes the original prompt $\mathcal{Q}$, the invalid output, and an error message $\mathcal{E}$ describing syntax errors or invalid asset references. The fix prompt is provided to the LLM to refine the task and repeat the process.

B. Benchmark Design

Inspired by Visual Question and Answering (VQA) benchmarks, our benchmark design enables enables evaluating specific competency axes:

Visual Competency: Assesses recognition of *color*, *semantics*, and *size*, capturing the policy's capability to link perceptual attributes with higher-level reasoning.

Procedural Competency: Evaluates the ability to perform tasks that involve action-oriented reasoning, including *affordances*, *reorientation*, or *stacking*.

Relational Competency: Tests understanding of language *conjunctions* (e.g., 'and', 'or'), *counting*, and *spatial* relationships, measuring how effectively the policy interprets multi-object instructions and scene structure.

Figure 4 shows examples of these questions accompanied by scene examples. Since any one task cannot be categorized as containing exactly evaluating one attribute, each task is labeled with one or more attributes belonging to one or more competencies.

In RoboLab, each task can be decomposed into a *sequential list of subtasks, each containing parallel events*. For example, the task *"Put the apple and orange on the plate, then put the banana in the bowl"* decomposes to subtasks `PickPlace(orange) & PickPlace(apple)` followed by subtask `PickPlace(banana)`, where each `PickPlace` contains parallel events (Grasp $\rightarrow$ Hover $\rightarrow$ Drop $\rightarrow$ Done) for each object.

Difficulty of tasks in our benchmarking system is rated using the following formula, based on the task length and require level of competency: $\text{DifficultyScore} = N_{\text{subtasks}} + \max(w_{\text{skill}})$ where w_{skill} is 0 for visual identification, 1 for spatial reasoning, 2 for procedural reasoning, and 3 for reorientation and dynamic tasks. We use this system based on how much reasoning and dexterity should be required in order for the task to be complete. Based on the difficulty score, tasks fall into one the following difficulty levels: *simple* (≤ 2), *moderate* (3–4), or *complex* (≥ 5). The difficulty levels allows the user to quickly analyze the results at a glance, although we provide a more granular analysis is possible by examining scene composition and task

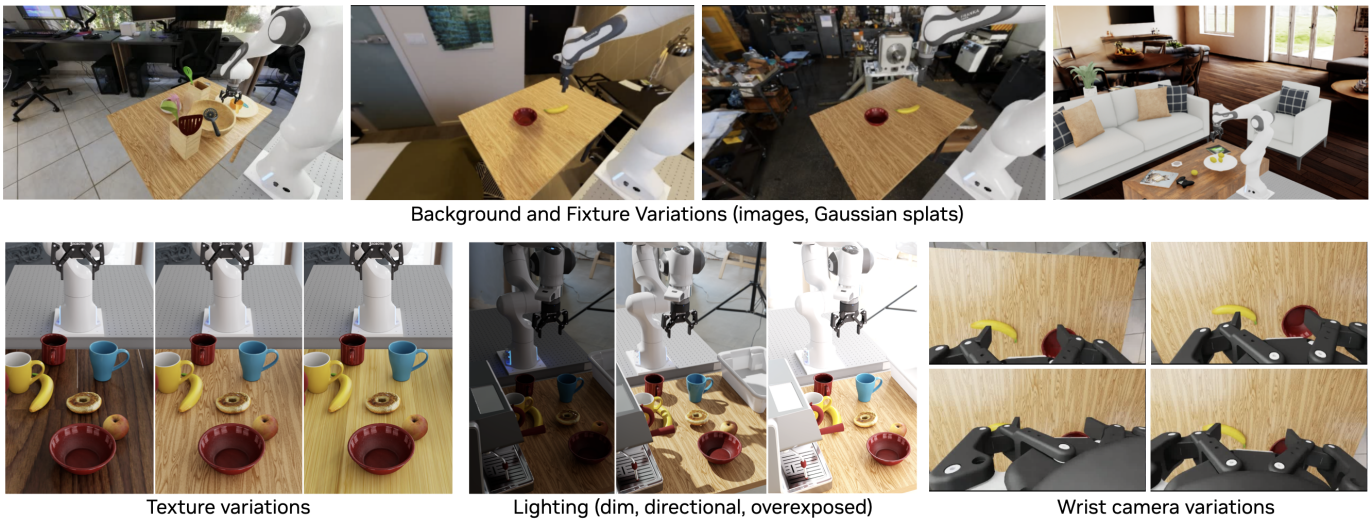


Fig. 6: Example scene variations, lighting variations, and camera pose variations in RoboLab.

horizon, discussed in Sec. IV-B.

C. Metrics for Evaluation

While task success rate remains a fundamental metric, prior work [13] has demonstrated that they fail to reveal nuanced aspects of policy behavior and failure modes. Unlike approaches relying on human judgment [11], we define a set of discrete and continuous metrics to characterize policy performance. All of the following measures are independent measures and together paint a complete picture of policy bias.

Normalized Scores. We compute a *normalized graded score* for each task based on the subtasks τ as follows: $Sc(\mathcal{T}) = \frac{1}{|\mathcal{T}|}\sum_{\tau\in\mathcal{T}} w_\tau Sc(\tau)$, where $Sc(\tau) = ||\sum_e w_e||$ is the subtask score given events e. By default, all events and subtask weights are equal. Subtask/event weights default to equal but are user-configurable per task, allowing consequential events (e.g., an initial grasp in a cluttered scene) to be weighted appropriately. Successful episodes will have a score of 1.

Language Variations. Tasks in RoboLab are paired with a set of language instructions that the evaluator can query from. Having a set of high-variance language instructions is crucial for task-generalist policies: Given language variants of the same underlying task, the policy should behave similarly. This provides insight into the reasoning failures.

Trajectory Metrics. Trajectory quality metrics capture characteristics of motion efficiency and optimality. We compute the following: *Spectral arc-length (SPARC)*, which evaluates motion smoothness [1] via the arc length of the normalized Fourier magnitude spectrum of the velocity profile. Given a speed profile $v(t)$ of the end effector over time interval $[0, T]$

$$\text{SPARC} = -\int_0^{\omega_c} \sqrt{\left(\frac{1}{\omega_c}\right)^2 + \left(\frac{d\hat{V}(\omega)}{d\omega}\right)^2}\, d\omega \qquad (1)$$

where $\hat{V}(\omega) = V(\omega)/V(0)$ represents the normalized Fourier magnitude spectrum. Smoother motions yield values closer to zero, while jerkier trajectories produce more negative values. We employ an adaptive cutoff frequency $\omega_c = \min(10 \text{ Hz}, \omega_\alpha)$, where $\omega_\alpha = \max_{k\in\mathcal{K}} \omega_k$ and $\mathcal{K} = \{k \mid \hat{V}(\omega_k) \geq \alpha\}$ denotes the set of frequency bins exceeding threshold $\alpha = 0.05$. This adaptive strategy ensures that the smoothness evaluation focuses on relevant frequency components. Lastly, trajectory optimality is assessed through end effector *speed* $v(t)$, and *path length* $l = \sum_{k=0}^{N-1} \|p_{k+1} - p_k\|$, where p_k denotes the end-effector position at timestep k. Shorter path lengths indicate more direct trajectories and generally reflect superior motion quality.

D. Sensitivity Analysis

We present a Bayesian framework for evaluating policy robustness across diverse environmental conditions using Simulation-Based Inference (SBI). This analysis provides insight into which scene parameters are most strongly linked to success and failure outcomes by learning an approximate posterior distribution over them given evaluation data. Let $\theta = (\theta^{\text{cont}}, \theta^{\text{disc}})$ denote the environment parameters comprising of continuous variables (e.g., object distance, camera displacement) and/or discrete variables. After evaluating policy π under varied conditions, we generate episodes $\mathcal{D} = \{(\theta_i, x_i)\}_{i=1}^N$ with observed outcomes x_i (e.g., task success). The posterior distribution $p(\theta \mid x) \propto p(x \mid \theta)p(\theta)$ is approximated using Mixed Neural Posterior Estimation (MNPE), which trains a neural density estimator $q_\phi(\theta \mid x)$ to directly learn the mapping from observations to parameter distributions. The resulting posterior $q_\phi(\theta \mid x)$ characterizes which scene variables are most associated with a target observation x. Our approach provides systematic assessment of which variables most strongly influence performance outcomes. Further details are in Appendix B.

Task Adherence. While task success is determined by the target world state, the policy may exhibit undesirable behaviors during rollout that do not affect the final outcome.

TABLE I: **Overall performance of SOTA policies on RoboLab.** While recent foundation models exhibit emerging capabilities across diverse task dimensions, overall performance remain limited. See Table IV and Table V for more granular breakdown of performance trajectory-quality results.

	Overall Metrics			**Difficulty** (succ% / score)			**Procedural** (succ% / score)			**Relational** (succ% / score)			**Visual** (succ% / score)		
Model	Succ% / Score (↑)	SPARC (↑)	Speed (↑)	simple	moderate	complex	affordance	reorientation	stacking	conjunction	counting	spatial	color	semantics	size
$\pi_{0.5}$ [26]	**28.0 / 0.43**	−8.34 ±6.7	**5.4** ±1.6	**29.7 / 0.39**	**31.5 / 0.51**	**13.5 / 0.44**	**10.0 / 0.27**	**28.3 / 0.48**	**35.0 / 0.57**	**43.8 / 0.55**	**65.7 / 0.75**	**21.0 / 0.36**	**23.8 / 0.42**	**23.2 / 0.39**	**30.0 / 0.36**
π_0-FAST [25]	15.5 / 0.27	−9.63 ±5.4	4.6 ±1.8	20.2 / 0.27	13.3 / 0.29	2.9 / 0.22	2.5 / 0.13	3.3 / 0.10	15.0 / 0.28	27.5 / 0.38	44.3 / 0.59	15.9 / 0.30	6.2 / 0.17	10.5 / 0.22	10.0 / 0.18
GR00T N1.6 [22]	7.2 / 0.17	**−6.87** ±4.6	4.3 ±1.4	8.8 / 0.14	7.9 / 0.25	0.0 / 0.12	0.8 / 0.11	0.0 / 0.03	8.3 / 0.18	10.0 / 0.14	18.6 / 0.34	7.2 / 0.22	3.8 / 0.16	6.8 / 0.17	0.0 / 0.03
π_0 [4]	5.0 / 0.12	−9.49 ±4.1	4.2 ±1.3	7.2 / 0.10	3.6 / 0.18	0.0 / 0.09	0.0 / 0.07	1.7 / 0.06	0.0 / 0.05	12.5 / 0.21	11.4 / 0.28	3.1 / 0.16	1.2 / 0.08	2.3 / 0.09	1.7 / 0.03
PaliGemma [3]	3.4 / 0.10	−16.52 ±10.2	1.9 ±1.6	3.4 / 0.06	4.9 / 0.16	0.0 / 0.09	0.0 / 0.01	1.7 / 0.07	0.0 / 0.06	3.8 / 0.06	22.9 / 0.36	1.7 / 0.12	0.8 / 0.09	3.5 / 0.09	0.0 / 0.05

For example, Fig. 3 demonstrates a successful episode in which the policy incorrectly grasped an extraneous object. Such errors highlight potential biases in the policy not captured by other metrics. Thus, capturing discrete events such as grasping wrong objects, executing redundant actions and unintended collisions, highlights reasoning and task adherence failures. Our benchmark automatically records instances of events; including wrong object grasped, object dropped, and gripper collisions.

IV. Experiments

We evaluate several off-the-shelf generalist robotics models, performing controlled ablations, and environmental perturbations to observe where failures concentrate. The experiments are designed to address the following questions: **Q1:** How well does today's SOTA models generalize, given varying language instructions, scene complexity, and environment pertubations? **Q2:** When and why does a policy fail? **Q3:** Can a simulated benchmark be used to evaluate real-world models?

A. Experiment Setup

We evaluate SOTA models on RoboLab-120, a benchmark containing 120 tasks of varying difficulty levels (65 simple, 38 moderate, 18 complex). We evaluate policies finetuned on the DROID robot [12], which is an open-sourced robot used for VLAs [9, 28]. DROID has a 7-DOF Franka Panda robot arm with a Robotiq-2F-85 gripper, an externally mounted ZED 2i camera with f=2.1mm, and a ZED mini as the wrist camera. All policies were fine-tuned on the DROID dataset [12]: $\pi_{0.5}$ [26], π_0-FAST [25], π_0 [4], PaliGemma [3], GR00T N1.6 [22].

The action space is 7-DOF Franka joint positions and a 1-DOF binary gripper command. The environments are composed of a default office-like background and natural lighting to mimic typical setups in the DROID dataset [12], with wrist and external camera poses designed to strongly match the real-world DROID robot. Each task was repeated with N=10 episodes per task. Notes on statistical analysis of N is discussed in Appendix A-A.

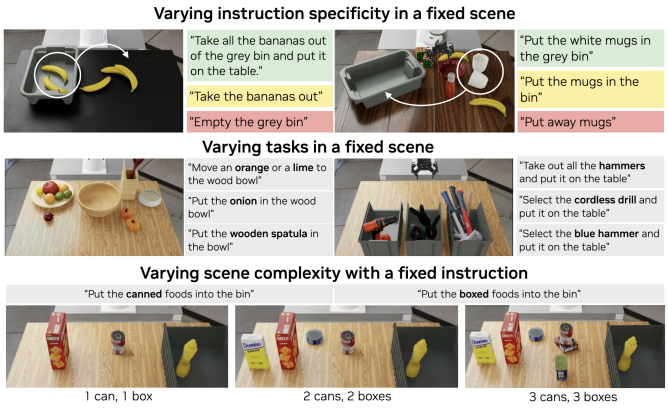


Fig. 7: Examples of language ablation experiments. Top: Same scene and goal, but the instruction wording ranges from precise to increasingly vague. Middle: Same scene, but the instruction specifies different tasks to perform. Bottom: Same instruction, but the scene becomes progressively more complex.

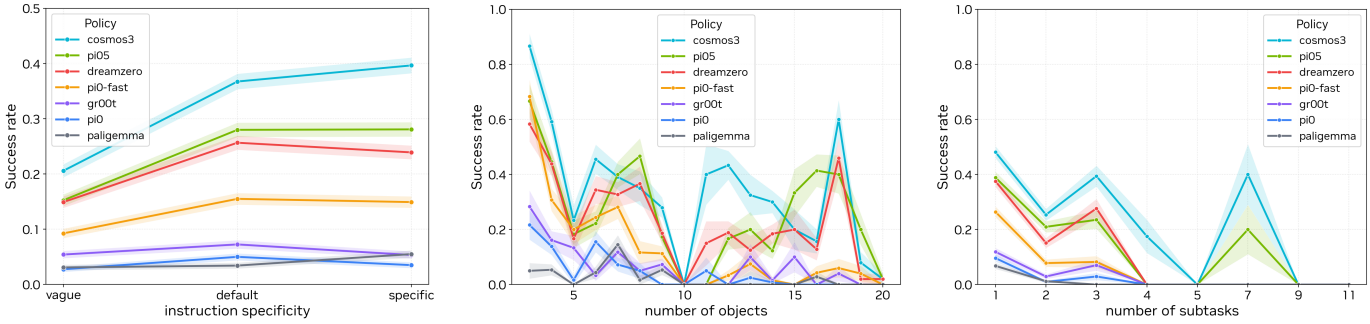
B. Task Results

Table I shows the overall results on RoboLab-120, highlighting success rate and score. Note that success rates indicates number of episodes that were completed, whereas the score indicates whether if the policy was able to make progress on any task. Overall success were low, which matches prior observations on out-of-domain generalization for generalist policies [36]. More interestingly, the score/success gap reveals capabilities that raw success rates obscure. $\pi_{0.5}$ achieves only 13.5% success on complex tasks yet attains a score of 0.44, indicating that nearly half of the partial-credit milestones are reached even when full task completion is rare. This indicates that contemporary policies often partially understand the task but fail in the final stages of execution.

Competency axes also highlighted asymmetric generalization in *relational* reasoning tasks. Overall, policies handled conjunctions and counting better than spatial relations. In *visual* grounding, performance remained low across attribute types, indicating brittle language-to-object binding beyond a narrow set of familiar object descriptions. *Procedural* understanding proved most challenging.

To further probe generalization robustness of the policies, we performed analysis on variations on instructions, scene complexity, and task difficulty. An example of these experiments is shown in Fig. 7.

Performance relative to instruction specificity. Fig. 8a reports overall RoboLab-120 success rates under three levels of instruction specificity (vague, default, specific) for the same underlying tasks and scenes. We observe degradation as instructions become more abstract (e.g., $\pi_{0.5}$ drops from 28.0% on default to 15.3% on vague) This sensitivity reveals that current models lack robust reasoning against the task goal. A truly generalist policy should be invariant to instruction phrasing. A policy that has understood the task should perform comparably whether the instruction is vague or specific.



(a) SR relative to instruction specificity (b) SR relative to scene complexity (c) SR relative to task horizon

Fig. 8: **Effect of language, scene complexity, and task horizon on policy performance (success rate).** Performance using success rate across complexity axes. For a robust policy, performance should relatively comparable as complexity increases. (a) Performance degrades as instructions become more abstract, indicating brittle task-level reasoning. (b) Performance degrades sharply as the number of objects in the scene increases, exposing systematic geometric biases in current models. (c) Performance degrades as the task horizon increases, indicating limited multi-step reasoning across all evaluated policies. Fig. 12 shows the same analysis with normalized scores for a more granular analysis.

Performance relative to scene complexity. Fig. 8b isolates the effect of scene complexity by increasing the number of objects visually present on the table, representing visual clutter. Success rates degrade as the number of objects in the scene increases for most policies; however, for $\pi_{0.5}$, some tasks were able to be achieved even with high visual clutter. These results indicates that current policies can perform some visual grounding in cluttered scenes; however, for some policies, additional distractors overwhelm the policy's ability to identify and manipulate the correct target.

Performance relative to task horizon. Fig. 8c shows that performance degrades as the task horizon increases. Performance degrades as the task horizon increases. This indicates that current models lack the multi-step reasoning and compositional planning required for longer-horizon tasks. A performance increase is observed at subtask=7 for $\pi_{0.5}$ on `CubesAndBlocksInBinTask` is further discussed in Appendix A-D.

Together, these results show how RoboLab isolates where generalization fails, addressing **Q1** supporting analysis that can inform data collection and model improvement priorities.

C. Sensitivity Analysis

We perform a set of variations given two basic tasks and observe the outcome, as illustrated in Fig. 6. These are: 1) Wrist and external camera poses; 2) object poses; 3) Visual features, including background and table textures; and 4) Lighting, including saturation and hue. Table II illustrates the results for all experiments.

Visual and Lighting variations. We vary the lighting conditions via color temperature shifts, lighting exposure and strong directional light that generates shadows as the robot is moving. **Lighting:** Models were robust to changes in lighting conditions, with 90–100% success across shadow variations, color temperature shifts, and 500× intensity changes. **Visual appearance:** Variations over 10 background textures and 4 table textures had minimal impact (<5% degradation), suggesting generalization to scene appearance changes.

Camera variation sensitivity analysis. We infer posteriors over camera displacement conditioned on task success (Fig. 9). Camera poses were randomized in both orientation and position for 10 episodes each. Displacement is calculated with respect to the nominal position of the cameras. Across all policies, the wrist-camera posterior is sharply concentrated near zero, indicating that successful execution often required the wrist camera to remain close to its nominal pose, while performance is more tolerant to external camera position changes. This indicates performance is critically dependent on wrist camera than external camera.

Object pose variation sensitivity analysis. We randomize initial object poses via a uniform distribution of 10cm, 20cm, and 30cm within its nominal placement (in front of the robot). We then infer posteriors over initial object poses conditioned on task success (see Fig. 9), relative to the robot pose. We observe a strong peak over 0.5m from the robot's origin, suggesting that objects placed at this distance has the highest probability of success, likely due to reachability.

Together, these results show how RoboLab's analysis framework can be used to systematically identify factors contributed to policy failure, addressing **Q2**.

D. Real-World Verification

To assess whether RoboLab can be used as a proxy for real-world evaluation, we compare performance results from RoboLab against RoboArena [28], an open-source real-world benchmarking system. Fig. 10 plots per-policy RoboLab-120 success rate against RoboArena Elo scores. We observe that the ranking between policies is strongly preserved (Spearman $\rho = 1.00$). Because RoboArena reports Elo while RoboLab reports success rates, Spearman rank correlation is more useful as a proxy, since it measures whether the two benchmarks induce the same ordering over policies. This indicates that RoboLab achieves benchmark-level correlation with real-world

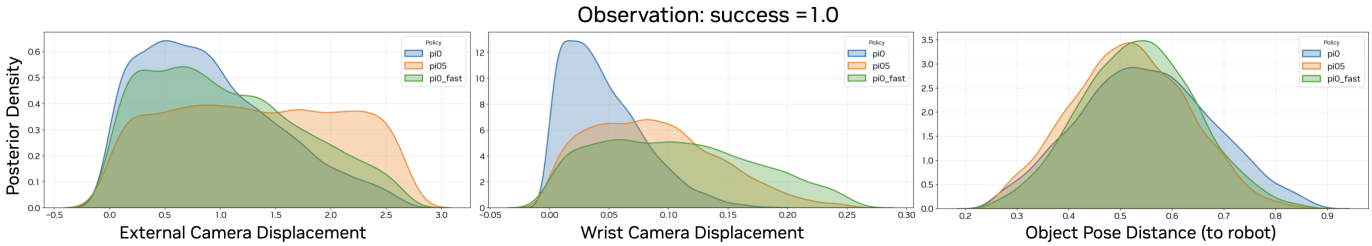


Fig. 9: Results of the sensitivity analysis using MNPE. Policies were highly sensitive to wrist-camera displacement from the nominal pose, indicating strong dependence on wrist-mounted camera calibration. Success also peaked for objects placed at approximately 0.5m from the robot, likely due to robot reachability.

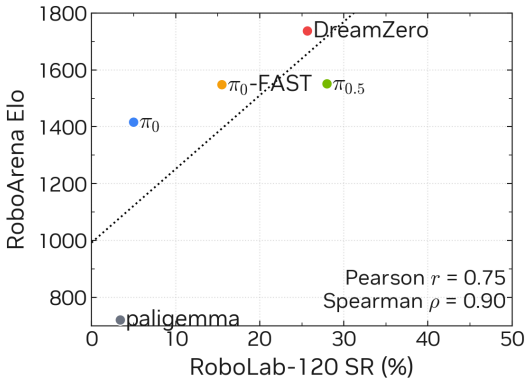


Fig. 10: Correlation between RoboLab-120 success rate and RoboArena Elo score across the four policies with both measurements available ($\pi_{0.5}$, π_0-FAST, π_0, PaliGemma). Rankings are preserved (Spearman $\rho = 1.00$) and the scores are positively correlated (Pearson $r = 0.68$), suggesting RoboLab-120 performance is a useful proxy for real-world policy quality.

TABLE II: Robustness to controlled environmental variations over two simple tasks (`BananaInBowl`, `BananaAndCubeInBowl`). PaliGemma is excluded as it fails to achieve meaningful results.

	$\pi_{0.5}$		π_0**-FAST**		π_0	
Variation	Succ.%	Time (s)	Succ.%	Time (s)	Succ.%	Time (s)
Lighting						
Color	96.7	14.5 ±7.9	93.3	17.9 ±10.7	6.7	31.1 ±4.3
Shadows	100.0	16.0 ±6.0	90.0	12.4 ±3.5	0.0	-
Dim	90.0	9.1 ±2.1	70.0	13.1 ±2.8	70.0	35.5 ±9.7
Overexposed	100.0	13.9 ±4.4	100.0	9.6 ±1.7	0.0	-
Visual Variations						
Background	85.0	14.4 ±8.7	70.0	21.3 ±10.7	25.0	31.6 ±11.8
Table texture	87.5	19.0 ±13.8	60.0	19.0 ±12.9	22.5	28.1 ±6.9
Object Pose						
10cm	95.0	16.2 ±9.2	55.0	26.5 ±13.8	22.5	34.7 ±13.3
20cm	95.0	19.7 ±10.2	40.0	21.8 ±9.5	20.0	37.4 ±11.2
30cm	62.5	18.9 ±8.7	35.0	24.3 ±11.9	17.5	24.3 ±11.9
Camera Pose						
external	85.0	17.4 ±11.3	45.0	27.7 ±10.6	50.0	27.4 ±16.0
wrist	60.0	21.9 ±13.4	25.0	20.1 ±9.9	10.0	35.3 ±10.4

performance, addressing **Q3**. We leave deeper investigation of task-level and motion-level correlation to future work.

V. Limitations

While RoboLab provides a flexible and scalable framework for evaluating language-conditioned manipulation, it currently focuses on rigid-body tabletop scenes and does not fully capture the challenges of deformable object manipulation (e.g., cloth, cables, bags). Moreover, many contact-rich skills that require precise force control, compliant interaction, or complex frictional dynamics are underrepresented and dependent on the physics simulation fidelity, limiting Robolab's coverage of fine-grained, low-level control tasks. Our subtask evaluation system breaks down for open-ended and ambiguous long-horizon tasks (e.g., "clean up all the laundry") and human judgment becomes necessary. However, given that current policies achieve fairly low scores, this is not yet a practical bottleneck, and our approach remains valid. Finally, although evaluation in high-fidelity simulation is a strong proxy for real-world performance, a residual visual distribution shift remains. This gap needs to be characterized further both by analyzing the behavior and robustness of the visual perception stack and through extensive validation on real-world deployments.

VI. Conclusion

Recent benchmarking efforts have made significant strides in scalable robot evaluation, but they primarily assess robustness to perturbations of training environments rather than true task generalization to novel scenarios. RoboLab addresses this gap by evaluating real world policies in a high-fidelity simulation, structured evaluation vectors that decompose policy competence into visual, procedural, and relational dimensions, and a set of sensitivity analysis set of novel analysis that provides insight into policy behavior for robotics. Our benchmarking framework enables the community to critically answer the question of *generalization* and *performance*. At the same time, the framework is designed to be pragmatically usable: new tasks can be authored in minutes by arranging objects on a tabletop and attaching language instructions, and a generative scene–task–environment workflow that supports continuous benchmark evolution.

References

[1] Sivakumar Balasubramanian, Alejandro Melendez-Calderon, and Etienne Burdet. A robust and sensitive metric for quantifying movement smoothness. *IEEE Transactions on Biomedical Engineering*, 59(8): 2126–2136, 2012. doi: 10.1109/TBME.2011.2179545.

[2] Prithviraj Banerjee, Sindi Shkodrani, Pierre Moulon, Shreyas Hampali, Shangchen Han, Fan Zhang, Linguang

Zhang, Jade Fountain, Edward Miller, Selen Basol, Richard Newcombe, Robert Wang, Jakob Julian Engel, and Tomas Hodan. HOT3D: Hand and object tracking in 3D from egocentric multi-view videos. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[3] Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, Daniel Salz, Maxim Neumann, Ibrahim Alabdulmohsin, Michael Tschannen, Emanuele Bugliarello, et al. Paligemma: A versatile 3b vlm for transfer, 2024. URL https://arxiv.org/abs/2407.07726.

[4] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A vision-language-action flow model for general robot control, 2024. URL https://arxiv.org/abs/2410.24164.

[5] Rishit Dagli, Donglai Xiang, Vismay Modi, Charles Loop, Clement Fuji Tsang, Anka He Chen, Anita Hu, Gavriel State, David I.W. Levin, and Maria Shugrina. Vomp: Predicting volumetric mechanical property fields. *arXiv preprint arXiv:2510.22975*, 2025. URL https://arxiv.org/abs/2510.22975.

[6] Jiayuan Gu, Fanbo Xiang, Xuanlin Li, Zhan Ling, Xiqiang Liu, Tongzhou Mu, Yihe Tang, Stone Tao, Xinyue Wei, Yunchao Yao, Xiaodi Yuan, Pengwei Xie, Zhiao Huang, Rui Chen, and Hao Su. Maniskill2: A unified benchmark for generalizable manipulation skills. *arXiv preprint arXiv:2302.04659*, 2023.

[7] Andrew Guo, Bowen Wen, Jianhe Yuan, Jonathan Tremblay, Stephen Tyree, Jeffrey Smith, and Stan Birchfield. HANDAL: A dataset of real-world manipulable object categories with pose annotations, affordances, and reconstructions. In *IROS*, 2023.

[8] Isaac Lab Team. Isaac Lab: A GPU-accelerated simulation framework for multi-modal robot learning. *arXiv preprint arXiv:2511.04831*, 2025. URL https://arxiv.org/abs/2511.04831.

[9] Arhan Jain, Mingtong Zhang, Kanav Arora, William Chen, Marcel Torne, Muhammad Zubair Irshad, Sergey Zakharov, Yue Wang, Sergey Levine, Chelsea Finn, Wei-Chiu Ma, Dhruv Shah, Abhishek Gupta, and Karl Pertsch. Polaris: Scalable real-to-sim evaluations for generalist robot policies, 2025. URL https://arxiv.org/abs/2512.16881.

[10] Stephen James, Zicong Ma, David R. Arrojo, and Andrew J. Davison. RLBench: The Robot Learning Benchmark & Learning Environment. *RAL*, 2020.

[11] Yash Jangir, Yidi Zhang, Kashu Yamazaki, Chenyu Zhang, Kuan-Hsun Tu, Tsung-Wei Ke, Lei Ke, Yonatan Bisk, and Katerina Fragkiadaki. Robotarena ∞: Scalable robot benchmarking via real-to-sim translation, 2025. URL https://arxiv.org/abs/2510.23571.

[12] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Robotics: Science and Systems (RSS)*, 2024.

[13] Hadas Kress-Gazit, Kunimatsu Hashimoto, Naveen Kuppuswamy, Paarth Shah, Phoebe Horgan, Gordon Richardson, Siyuan Feng, and Benjamin Burchfiel. Robot learning as an empirical science: Best practices for policy evaluation, 2024. URL https://arxiv.org/abs/2409.09491.

[14] Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabrael Levine, Wensi Ai, Benjamin Martinez, et al. Behavior-1k: A human-centered, embodied ai benchmark with 1,000 everyday activities and realistic simulation. *arXiv preprint arXiv:2403.09227*, 2024.

[15] Xuanlin Li, Kyle Hsu, Jiayuan Gu, Karl Pertsch, Oier Mees, Homer Rich Walke, Chuyuan Fu, Ishikaa Lunawat, Isabel Sieh, Sean Kirmani, Sergey Levine, Jiajun Wu, Chelsea Finn, Hao Su, Quan Vuong, and Ted Xiao. Evaluating real-world robot manipulation policies in simulation. *arXiv preprint arXiv:2405.05941*, 2024.

[16] Yunzhi Lin, Jonathan Tremblay, Stephen Tyree, Patricio A. Vela, and Stan Birchfield. Multi-view fusion for multi-level robotic scene understanding. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 6817–6824, 2021. doi: 10.1109/IROS51168.2021.9635994.

[17] Zhiqiu Lin, Deepak Pathak, Baiqi Li, Jiayao Li, Xide Xia, Graham Neubig, Pengchuan Zhang, and Deva Ramanan. Evaluating text-to-visual generation with image-to-text generation, 2024. URL https://arxiv.org/abs/2404.01291.

[18] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.

[19] Oier Mees, Lukas Hermann, Erick Rosete-Beas, and Wolfram Burgard. CALVIN: A Benchmark for Language-Conditioned Policy Learning for Long-Horizon Robot Manipulation Tasks. *IEEE Robotics and Automation Letters*, 7(3):7327–7334, 2022.

[20] Nicolas Moenne-Loccoz, Ashkan Mirzaei, Or Perel, Riccardo de Lutio, Janick Martinez Esturo, Gavriel State, Sanja Fidler, Nicholas Sharp, and Zan Gojcic. 3d gaussian ray tracing: Fast tracing of particle scenes. *ACM Transactions on Graphics and SIGGRAPH Asia*, 2024.

[21] NVIDIA. Isaac Sim. URL https://github.com/isaac-sim/IsaacSim.

[22] NVIDIA. GR00T N1: An open foundation model for generalist humanoid robots. In *ArXiv Preprint*, March 2025.

[23] OpenAI. Gpt-4 technical report, 2024. URL https://arxiv.org/abs/2303.08774.

[24] OpenAI. Openai o1 system card, 2024. URL https://arxiv.org/abs/2412.16720.

[25] Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for

vision-language-action models, 2025. URL https://arxiv.org/abs/2501.09747.

[26] Physical Intelligence. $\pi_{0.5}$: a vision-language-action model with open-world generalization, 2025. URL https://arxiv.org/abs/2504.16054.

[27] Mohammad Nomaan Qureshi, Sparsh Garg, Francisco Yandun, David Held, George Kantor, and Abhisesh Silwal. Splatsim: Zero-shot sim2real transfer of rgb manipulation policies using gaussian splatting, 2024. URL https://arxiv.org/abs/2409.10161.

[28] RoboArena Team. Roboarena: Distributed real-world evaluation of generalist robot policies. In *Proceedings of the Conference on Robot Learning (CoRL 2025)*, 2025.

[29] Martin Sedlacek, Pavlo Yefanov, Georgy Ponimatkin, Jai Bardhan, Simon Pilc, Mederic Fourmy, Evangelos Kazakos, Cees G. M. Snoek, Josef Sivic, and Vladimir Petrik. REALM: A real-to-sim validated benchmark for generalization in robotic manipulation. *arXiv preprint arXiv:2512.19562*, 2025. URL https://arxiv.org/abs/2512.19562.

[30] TRI LBM Team. A careful examination of large behavior models for multitask dexterous manipulation, 2025. URL https://arxiv.org/abs/2507.05331.

[31] Qi Wu, Janick Martinez Esturo, Ashkan Mirzaei, Nicolas Moenne-Loccoz, and Zan Gojcic. 3dgut: Enabling distorted cameras and secondary rays in gaussian splatting. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[32] Yu Xiang, Tanner Schmidt, Venkatraman Narayanan, and Dieter Fox. PoseCNN: A convolutional neural network for 6D object pose estimation in cluttered scenes. In *Robotics: Science and Systems (RSS)*, 2018.

[33] Yandan Yang, Baoxiong Jia, Shujie Zhang, and Siyuan Huang. Sceneweaver: All-in-one 3d scene synthesis with an extensible and self-reflective agent, 2025. URL https://arxiv.org/abs/2509.20414.

[34] Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Avnish Narayan, Hayden Shively, Adithya Bellathur, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning, 2021. URL https://arxiv.org/abs/1910.10897.

[35] Kaifeng Zhang, Shuo Sha, Hanxiao Jiang, Matthew Loper, Hyunjong Song, Guangyan Cai, Zhuo Xu, Xiaochen Hu, Changxi Zheng, and Yunzhu Li. Real-to-sim robot policy evaluation with gaussian splatting simulation of soft-body interactions. *arXiv preprint arXiv:2511.04665*, 2025.

[36] Xueyang Zhou, Yangming Xu, Guiyao Tie, Yongchao Chen, Guowen Zhang, Duanfeng Chu, Pan Zhou, and Lichao Sun. Libero-pro: Towards robust and fair evaluation of vision-language-action models beyond memorization. *arXiv preprint arXiv:2510.03827*, 2025.

[37] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. In *arXiv preprint arXiv:2009.12293*, 2020.

[38] Alex Zook, Fan-Yun Sun, Josef Spjut, Valts Blukis, Stan Birchfield, and Jonathan Tremblay. Grs: Generating robotic simulation tasks from real-world images. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 594–603, 2025.

ACKNOWLEDGEMENTS

We thank Arhan Jain, Karl Pertsch, Alperen Degirmenci, Valts Blukis, and Siyi Chen for helpful discussions in building RoboLab.

APPENDIX A
DETAILS ON THE ROBOLAB BENCHMARK

In this section we provide detail on the benchmark.

RoboLab provides a set of ∼300 object assets from well-known 3D pose estimation benchmarks; including YCB [32], HOT3D [2], HOPE [16], HANDAL [7], and VoMP [5]. Each asset contains a visual and collision mesh, with mass and friction properties added. Each object has a language description and an object label attached to it. This forms the catalog of objects used in the scenes, in either manual or LLM-scaled scene environments.

Along with RoboLab, we introduce an initial benchmark, RoboLab-120, which contains 120 manually generated tasks. Details on the scenes and tasks, including images and language instructions is available along with full benchmark results is available on the open source repository https://github.com/NVlabs/RoboLab/.

A. Statistical significance of results

Due to stochastic nature of simulators and the policies, it is recommended to run multiple episodes per task per policy [30]. We evaluate each policy on N=10 episodes per task, and report results on our benchmark as aggregate results instead of per-task analysis between policies.

With only 10 trials, the 95% confidence interval on a single *per-task* success rate is approximately ±30% near p=0.5 and ±19% near p=0.9, meaning per-task numbers should be interpreted as coarse indicators rather than precise estimates. Aggregate scores across all tasks are considerably tighter, since the effective sample size scales with the number of tasks, but per-task claims and fine-grained policy comparisons remain underpowered. For results that support per-task conclusions or resolve differences smaller than 10% between policies, we recommend increasing N to at least 100 episodes per task, which reduces the 95% CI half-width to roughly ±10% at p=0.5 and ±6% at p=0.9, giving meaningful results for per-task analysis between policies.

B. Comparison against other benchmarks

We describe a comparison of existing methods for task generation, which we categorize into *high vs low-overhead* in the following table.

All three demand considerable manual effort per scene, making large-scale scene/task creation tedious. In contrast, RoboLab lets users compose scenes via drag-and-drop, and tasks via manual text specification or invoke an AI agent to generate N tasks per scene at once.

TABLE III: Comparison of task creation pipelines across benchmarks. O/H denotes overhead.

Benchmark	Pipeline (per 1 scene 1 task)	O/H
RobotArena ∞	VLM+2D bbox detector per frame→2D-to-3D diffusion→3D asset retrieval→diff. rendering	High
Polaris	RGB-D video→camera calib.→3DGS→mesh extraction, semantic segmentation→physics	High (1hr)
LIBERO	Object+placement+task+goal (python)→BDDL	High
RoboLab	**Drag-and-drop→text instructions**	**Low**

(a) Initial scene (b) succ. eps. 1 (c) succ. eps. 2

Fig. 11: Initial scene and final-frame viewports from two successful $\pi_{0.5}$ rollouts on `CubesAndBlocksInBinTask` (7 subtasks). While the task-horizon is long, the task itself is non-causal. The policy was able to make significant progress and completion on this task.

C. Score across complexity axes

For completeness, we illustrate Fig. 8 but with policy score instead of success in Fig. 12. The gap between the two metrics is largest on *complex* tasks: $\pi_{0.5}$ drops to 13.5% success but retains a score of 0.44. On complex tasks, $\pi_{0.5}$ still earns 0.35 of the milestones even within failed episodes, indicating that the policy reaches a substantial fraction of subgoals before failing. The same pattern holds in procedural categories where full completion is rare but partial progress is not.

D. Anomalous long-horizon success at subtask=7

Fig. 8c shows a non-monotonic spike at subtask=7: $\pi_{0.5}$ recovers to ∼20% success and a score of 0.68 at this horizon, despite achieving 0% at neighboring horizons. The results show that the subtask=7 bin is dominated by `CubesAndBlocksInBinTask` (Fig. 11). This task contains repetitive similar pick-and-place sequences with geometrically simple objects (cubes), instead of a true causal long-horizon task where multi-step reasoning is required. This suggests the apparent "recovery" at subtask=7 reflects task composition rather than improved long-horizon reasoning. Fig. 8c should therefore be read with this caveat in mind, not as a clean horizon-vs-difficulty curve.

APPENDIX B
DETAILS OF MNPE SENSITIVITY ANALYSIS

MNPE allows us to analyze the relationship between scene parameters and policy outcomes in a likelihood-free Bayesian inference setting.

Variable Definitions. Let $\theta \in \Theta$ denote the vector of variation parameters. In our camera pose sensitivity experiments,

TABLE IV: **Capability metrics** per task category on RoboLab. "Succ%" is the fraction of episodes that fully complete the task; "Score" is the mean per-episode normalized score awarded for partial subtask completion (Sec. III-C); "Score (fail)" is the mean score of failed episodes only, which isolates partial progress on episodes that did not complete. Trajectory-quality metrics (SPARC, Speed) for the same categories are reported in Table V.

Task Categories	#	$\pi_{0.5}$			π_0-FAST			GR00T N1.6			π_0			PaliGemma		
		Succ %	Score	Score (fail)	Succ %	Score	Score (fail)	Succ %	Score	Score (fail)	Succ %	Score	Score (fail)	Succ %	Score	Score (fail)
Total	120	28.0%	0.43	0.21	15.5%	0.27	0.13	7.2%	0.17	0.11	5.0%	0.12	0.08	3.4%	0.10	0.07
simple	64	29.7%	0.39	0.13	20.2%	0.27	0.08	8.8%	0.14	0.05	7.2%	0.10	0.03	3.4%	0.06	0.03
moderate	39	31.5%	0.51	0.28	13.3%	0.29	0.18	7.9%	0.25	0.19	3.6%	0.18	0.15	4.9%	0.16	0.12
complex	17	13.5%	0.44	0.35	2.9%	0.22	0.20	0.0%	0.12	0.12	0.0%	0.09	0.09	0.0%	0.09	0.09
Procedural	22	21.8%	0.42	0.25	6.4%	0.16	0.11	2.7%	0.11	0.09	0.5%	0.07	0.06	0.5%	0.04	0.04
affordance	12	10.0%	0.27	0.19	2.5%	0.13	0.10	0.8%	0.11	0.10	0.0%	0.07	0.07	0.0%	0.01	0.01
reorientation	6	28.3%	0.48	0.27	3.3%	0.10	0.07	0.0%	0.03	0.03	1.7%	0.06	0.04	1.7%	0.07	0.05
stacking	6	35.0%	0.57	0.35	15.0%	0.28	0.16	8.3%	0.18	0.10	0.0%	0.05	0.05	0.0%	0.06	0.06
Relational	42	33.8%	0.47	0.20	23.3%	0.37	0.17	10.0%	0.23	0.15	6.4%	0.19	0.13	5.7%	0.16	0.11
conjunction	8	43.8%	0.55	0.21	27.5%	0.38	0.14	10.0%	0.14	0.05	12.5%	0.21	0.10	3.8%	0.06	0.03
counting	7	65.7%	0.75	0.28	44.3%	0.59	0.27	18.6%	0.34	0.18	11.4%	0.28	0.18	22.9%	0.36	0.16
spatial	29	21.0%	0.36	0.19	15.9%	0.30	0.17	7.2%	0.22	0.16	3.1%	0.16	0.13	1.7%	0.12	0.11
Visual	84	23.8%	0.39	0.20	8.8%	0.20	0.12	5.0%	0.15	0.11	1.9%	0.08	0.06	2.7%	0.09	0.07
color	26	23.8%	0.42	0.23	6.2%	0.17	0.11	3.8%	0.16	0.13	1.2%	0.08	0.07	0.8%	0.09	0.09
semantics	60	23.2%	0.39	0.20	10.5%	0.22	0.13	6.8%	0.17	0.10	2.3%	0.09	0.07	3.5%	0.09	0.06
size	6	30.0%	0.36	0.09	10.0%	0.18	0.09	0.0%	0.03	0.03	1.7%	0.03	0.01	0.0%	0.05	0.05

TABLE V: **Trajectory-quality metrics** per task category on RoboLab. "SPARC" is the spectral arc length of the end-effector velocity profile (Sec. III-C)—smoother motions yield values closer to zero, jerkier motions are more negative. "Speed" is the mean end-effector speed in cm/s. Standard deviations across episodes are shown in gray. Capability metrics for the same categories are reported in Table IV.

Task Categories	#	$\pi_{0.5}$		π_0-FAST		GR00T N1.6		π_0		PaliGemma	
		SPARC	Speed (cm/s)	SPARC	Speed (cm/s)	SPARC	Speed (cm/s)	SPARC	Speed (cm/s)	SPARC	Speed (cm/s)
Total	120	−8.34 (±6.65)	5.4 (±1.6)	−9.63 (±5.40)	4.6 (±1.8)	−6.87 (±4.63)	4.3 (±1.4)	−9.49 (±4.12)	4.2 (±1.3)	−16.52 (±10.21)	1.9 (±1.6)
simple	64	−7.38 (±6.66)	5.4 (±1.4)	−8.09 (±4.41)	4.7 (±1.7)	−6.37 (±5.00)	4.5 (±1.4)	−8.46 (±4.37)	4.3 (±1.4)	−15.33 (±8.73)	2.0 (±1.7)
moderate	39	−8.85 (±7.21)	5.4 (±1.8)	−10.07 (±5.44)	4.7 (±2.1)	−6.92 (±4.21)	4.2 (±1.3)	−10.13 (±2.99)	4.1 (±1.3)	−15.02 (±7.52)	2.1 (±1.7)
complex	17	−10.83 (±3.89)	4.8 (±1.3)	−14.38 (±5.75)	4.1 (±1.3)	−8.63 (±3.52)	4.1 (±1.4)	−11.91 (±4.09)	3.8 (±1.1)	−24.46 (±15.68)	1.5 (±0.8)
Procedural	22	−10.08 (±4.93)	4.8 (±1.4)	−11.06 (±6.31)	4.3 (±1.4)	−6.98 (±5.16)	3.8 (±1.3)	−10.58 (±2.96)	4.1 (±1.1)	−16.07 (±7.69)	1.5 (±0.8)
affordance	12	−10.80 (±4.47)	4.1 (±1.3)	−11.74 (±5.97)	3.6 (±1.0)	−6.79 (±2.49)	3.9 (±1.2)	−11.12 (±3.20)	4.0 (±1.2)	−15.40 (±8.50)	1.5 (±0.8)
reorientation	6	−10.08 (±3.69)	4.8 (±1.2)	−11.87 (±7.80)	4.4 (±1.4)	−7.05 (±1.64)	3.2 (±0.8)	−10.97 (±2.64)	3.9 (±0.9)	−17.63 (±8.49)	1.3 (±0.5)
stacking	6	−9.04 (±5.99)	5.9 (±1.1)	−9.05 (±4.12)	5.4 (±1.2)	−7.41 (±9.13)	4.3 (±1.5)	−9.35 (±2.28)	4.1 (±0.9)	−16.72 (±6.59)	1.4 (±0.9)
Relational	42	−7.50 (±3.95)	5.5 (±1.6)	−8.70 (±4.81)	4.9 (±2.1)	−6.51 (±2.48)	4.3 (±1.3)	−8.87 (±2.92)	4.5 (±1.4)	−14.93 (±10.38)	2.4 (±2.1)
conjunction	8	−6.88 (±4.04)	5.7 (±1.4)	−7.96 (±3.97)	5.0 (±1.6)	−7.47 (±3.05)	3.6 (±1.2)	−7.74 (±2.07)	4.8 (±1.2)	−13.77 (±9.36)	2.4 (±1.9)
counting	7	−7.16 (±4.91)	6.7 (±2.3)	−9.43 (±5.95)	5.9 (±3.1)	−7.64 (±2.62)	4.0 (±1.2)	−9.61 (±2.60)	4.2 (±1.0)	−9.88 (±5.49)	3.7 (±2.7)
spatial	29	−8.17 (±3.90)	5.0 (±1.3)	−8.99 (±4.76)	4.5 (±1.8)	−6.06 (±2.06)	4.6 (±1.3)	−9.18 (±3.16)	4.5 (±1.5)	−16.42 (±10.84)	2.0 (±1.8)
Visual	84	−8.73 (±7.39)	5.3 (±1.6)	−10.52 (±5.60)	4.3 (±1.7)	−7.10 (±4.71)	4.3 (±1.3)	−9.81 (±3.49)	3.9 (±1.2)	−17.72 (±10.98)	1.8 (±1.4)
color	26	−7.93 (±4.31)	5.2 (±1.5)	−9.75 (±4.91)	4.3 (±1.3)	−6.74 (±5.72)	4.1 (±1.3)	−9.30 (±2.83)	3.9 (±1.2)	−16.43 (±10.28)	2.0 (±1.5)
semantics	60	−9.26 (±8.44)	5.4 (±1.6)	−10.96 (±5.74)	4.2 (±1.8)	−7.32 (±5.35)	4.4 (±1.3)	−10.18 (±3.70)	4.0 (±1.3)	−17.99 (±11.20)	1.8 (±1.4)
size	6	−6.85 (±2.34)	5.4 (±1.4)	−7.82 (±3.53)	4.4 (±1.3)	−7.30 (±2.20)	4.3 (±1.7)	−7.77 (±2.76)	3.7 (±1.1)	−17.76 (±7.61)	1.7 (±0.9)

$\theta = (d_{\text{ext}}, d_{\text{wrist}}) \in \mathbb{R}^2$ where d_{ext} and d_{wrist} represent the displacement of the external and wrist cameras from their reference configurations, respectively, in SE(3).

Let $x \in \{0, 1\}$ denote the binary task success indicator, and let π denote the robot policy being evaluated.

Handling Mixed Parameters. For experiments involving both continuous parameters (e.g., pose distances) and discrete parameters (e.g., lighting levels, table materials), MNPE handles mixed continuous-discrete parameters through factorization: $q_\phi(\theta \mid x) = q_\phi(\theta^{\text{cont}} \mid \theta^{\text{disc}}, x) \cdot q_\phi(\theta^{\text{disc}} \mid x)$, where discrete components use softmax distributions and continuous components use normalizing flows. In our camera pose experiments, all parameters are continuous.

Pose Distance Metric. Poses are represented as 7-DoF transformations $\mathbf{T} = (\mathbf{p}, \mathbf{q})$ comprising position $\mathbf{p} \in \mathbb{R}^3$ and unit quaternion orientation $\mathbf{q} \in \mathbb{H}$. We compute a weighted distance from the reference configuration:

$$d(\mathbf{T}, \mathbf{T}_{\text{ref}}) = \|\mathbf{p} - \mathbf{p}_{\text{ref}}\|_2 + \beta \cdot d_{\text{SO(3)}}(\mathbf{q}, \mathbf{q}_{\text{ref}}), \tag{2}$$

where the geodesic distance on SO(3) is:

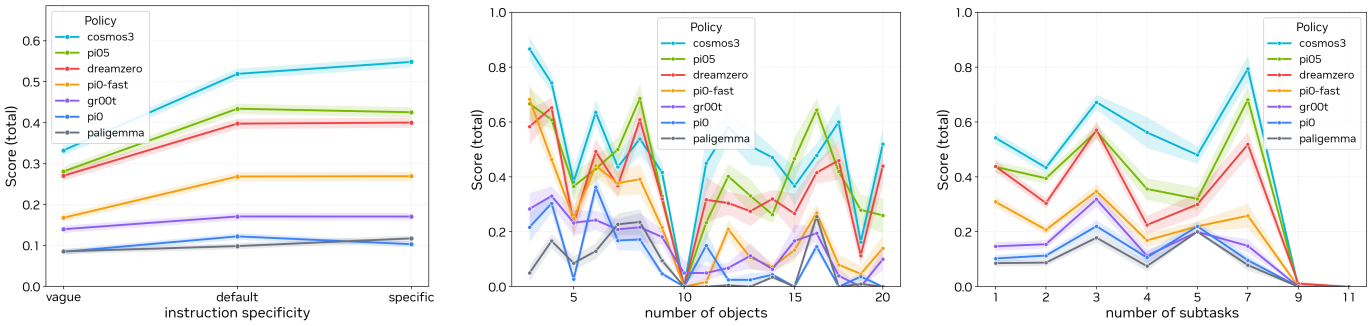
$$d_{\text{SO(3)}}(\mathbf{q}_1, \mathbf{q}_2) = 2 \arccos\left(\min(1, |\mathbf{q}_1 \cdot \mathbf{q}_2|)\right). \tag{3}$$

The weighting factor $\beta = 1.0$ balances translational (meters) and rotational (radians) components. For camera displacement, reference poses correspond to nominal camera mounting positions. For object pose, reference pose is the origin of the robot base.

Prior Specification. We adopt non-informative uniform priors to avoid biasing the inference toward any particular parameter region. For continuous parameters normalized to the unit interval:

$$p(\theta) = \prod_{j=1}^{m} \text{Uniform}(0, 1) = 1, \quad \theta_j \in [0, 1]. \tag{4}$$

Training Objective. The neural network parameters ϕ are optimized by minimizing the negative log-likelihood over the



(a) Score relative to instruction specificity (b) Score relative to scene complexity (c) Score relative to task horizon

Fig. 12: **Score (partial credit) across complexity axes.** Same complexity axes as Fig. 8, but plotting score (which awards partial credit for failed episodes) instead of success rate. Shaded bands show SE. Score degrades more gracefully than strict success, indicating policies make meaningful progress even when they fail to fully complete the task.

training dataset, $\mathcal{D} = \{(\theta_i, x_i)\}_{i=1}^{N}$:

$$\mathcal{L}(\phi) = -\frac{1}{N}\sum_{i=1}^{N} \log q_\phi(\theta_i \mid x_i). \tag{5}$$

We train for 50 epochs using the Adam optimizer on data collected from the camera pose variation and initial pose variation experiments (see Table II).

Importance Sampling Correction. Since the experimental data may sample parameters non-uniformly, we apply importance sampling to recover the posterior under a uniform prior: $p(\theta \mid x) \approx \frac{p(\theta)}{\tilde{p}(\theta)} q_\phi(\theta \mid x)$, where $\tilde{p}(\theta)$ is the empirical proposal distribution. We correct posterior samples using importance weights:

$$w_i = \frac{p(\theta_i)}{\tilde{p}(\theta_i)}, \tag{6}$$

where $\tilde{p}(\theta)$ is estimated via Gaussian kernel density estimation on the training data. The effective sample size $\text{ESS} = 1/\sum_i \bar{w}_i^2$ quantifies the efficiency of this correction.

Posterior Inference. Given a query observation x_o (e.g., $x_o = 1$ for successful task completion), we draw $N_s = 5000$ samples from the learned posterior:

$$\left\{\theta^{(i)}\right\}_{i=1}^{N_s} \sim q_\phi(\theta \mid x_o). \tag{7}$$

Posterior Statistics. For each continuous parameter, we compute the posterior mean and 95% credible interval:

$$\hat{\mu}_j = \frac{1}{N_s}\sum_{i=1}^{N_s} \theta_j^{(i)}, \tag{8}$$

$$\text{CI}_{95\%}^{(j)} = \left[Q_{0.025}\left(\{\theta_j^{(i)}\}\right),\ Q_{0.975}\left(\{\theta_j^{(i)}\}\right)\right], \tag{9}$$

where $Q_\alpha(\cdot)$ denotes the α-quantile.

This analysis reveals which variation parameters are most strongly associated with successful task outcomes: a posterior distribution tightly concentrated near zero indicates high sensitivity to that parameter (the policy requires it to remain near the reference value), while a broad posterior indicates robustness to variation.

APPENDIX C
DETAILS ON SCALING SCENE GENERATION

We present additional implementation details on scene generation (Section III-A1).

Fig. 13: (*Left*) We show one of our Gaussian Splat + Mesh scenes in RoboLab. This scene has a Gaussian splat background with a collision mesh for the splat estimated with 3DGRUT [20, 31], and a mesh foreground. All objects in the scene have spatially varying density, and thus mass is estimated with VoMP [5]. (*Right*) We show a VLA running a task in this scene.

A. Stage I: Predicates for Semantic Planning

The following predicates are used:

- `PlaceIn`(x, y): Object x must be contained within y (e.g., fruit in a bowl).
- `PlaceOn`(x, y): Object x is supported by y (e.g., mug on a coaster).
- `ClusterAround`$(x, \{y_i\})$: Object x acts as an anchor for a group $\{B_i\}$.
- `PlaceAnywhere`(x): Object x is placed freely on the global support surface (table).

If a predicate refers to a non-existent anchor, it is downgraded to a `PlaceAnywhere` constraint to preserve the object in the scene.

B. Stage II: Geometric Constraint Solving

For **Global Placement** (`PlaceAnywhere`), we utilize rejection sampling on the global table surface bounds, checking

collision against all currently placed objects using SAT on OBBs. To handle high-density scenes, we employ two strategies: (1) an adaptive relaxation loop that progressively increases collision margins if a valid layout is not found, and (2) a stochastic perturbation step that randomly jitters all object positions when the solver converges to a local minimum (Algorithm 1).

For **Stacking** (`PlaceOn`$(b_i, b_{support})$), we sample positions on the top surface of $b_{support}$ using rejection sampling (up to $K = 20$ attempts) to find a position $\mathbf{p}_{xy}$ such that $\text{OBB}(b_i) \cap \text{OBB}(b_{existing}) = \emptyset$ for all previously placed objects on the same support (Algorithm 2).

For **Containment** (`PlaceIn`$(b_i, b_{container})$), we compute the available interior volume of $b_{container}$ using its bounding box dimensions $\mathbf{d}_b$. We employ a packing heuristic that discretizes the container's floor into a grid with resolution $s = \max(\mathbf{d}_i^x, \mathbf{d}_i^y) + \epsilon_{margin}$. A cell (u, v) is considered valid if it is unoccupied and within the container's bounds scaled by factor $\gamma = 0.7$ to avoid edge collisions. We assign o_i to the first valid cell, setting its height $z_i = z_{container} + h_{container}/2$.

C. Baseline Method

To validate the efficacy of our hierarchical approach, we implement a robust baseline inspired by standard domain randomization techniques. The baseline operates in a single pass without iterative feedback. The process begins with the LLM selecting a list of objects O and suggesting a grid layout (rows $R\times$ columns C) for the table surface. The table surface is then divided into $R \times C$ rectangular cells, and objects are assigned to cells sequentially. Within each cell k, the object's position is jittered uniformly: $\mathbf{p}_{xy} \sim \mathcal{U}(\text{center}_k - w/4, \text{center}_k + w/4)$. This ensures basic separation but precludes complex stacking or containment, as objects are simply placed at a safe height $z = z_{table} + h_{obj}/2$. Finally, we run the same physics simulation pass as in our method to allow objects to settle under gravity, resolving minor inter-penetrations but without the capability to correct semantic or structural failures.

D. Experiments

We compare our scene generation with the baseline method using popular scene generation metrics, VQA score [17], GPT preference where it is shown two images each from the baseline or our method and is asked to pick one, and following Yang et al. [33], we report the visual realism (Real.), functionality (Func.), layout correctness (Lay.), Quality (Qual.) and scene completeness (Comp.) scores. We use GPT-4o [23] to generate scenes from our method and baseline; and use GPT-4.1 [23] for the evaluations. These metrics are computed on rendered RGB images from two viewpoints: a frontal view aligned with the table axis (camera at $(1.0, 0.0, 0.7)$ looking at table center) and an angled perspective view (camera at $(-0.3, 0.3, 0.7)$).

We show quantitative comparisons across 100 generated scenes for our method compared to the baselines in Table VI. We show the quantitative comparisons split by the number of objects in the scene ($[1, 5]$ objects, $[6, 15]$ objects, and $[16, 20]$ objects) in Table VII. We show the quantitative comparisons split across the 10 scene themes we use in Table VIII. We find our method consistently outperforms the baseline across all metrics, with particularly large gains in visual realism and semantic functionality.

Algorithm 1 Spatial Constraint Solver

Input: Objects B, Predicates P, Table Bounds L_{max}
Output: 2D coordinates (x, y, θ) for all base objects

Margins $M \leftarrow [\mu, 1.25\mu, 1.5\mu, 2.0\mu]$
for all margin $\in M$ **do**
  {Phase 1: Initialization}
  Randomize (x, y) for all loose objects inside L_{max}
  for all $p \in P$ **do**
    if p.type == place-on-base **then**
      p.object.$(x, y, \theta) \leftarrow (p.x, p.y, p.\text{yaw})$
    else if p.type == cluster-around **then**
      PolarPlace(p.targets, p.anchor, p.radius)
    end if
  end for
  {Phase 2: Relative Constraints}
  while constraints not satisfied **do**
    ApplyRelativeConstraints(P)
  end while
  ApplyOrientations(P)
  {Phase 3: Collision Resolution}
  for $k = 1$ **to** K_{max} **do**
    $C \leftarrow$ FindCollisions(B, margin)
    if $C = \emptyset$ **then**
      return Success
    end if
    if $|C|$ not decreasing for 10 steps **then**
      PerturbPositions(B)
    end if
    for all $(o_i, o_j) \in C$ **do**
      ResolveOverlap(b_i, b_j, margin)
      ClampToBounds(b_i, b_j, L_{max})
    end for
  end for
end for
return Failure

APPENDIX D
DETAILS ON TASK GENERATION EVALUATION

We evaluated the quality of our task generation method using an LLM-as-judge framework. Tasks were generated by prompting an LLM (o1 [24]) with scene descriptions and our Category templates[1]. For each generated task, we extracted the natural language instruction and the corresponding termination conditions (success criteria implemented as predicate functions) through static analysis of the generated Python code. We then prompted an LLM (also o1 [24]) to assess alignment between

[1]For 2 simple scenes, we generated 1 task for each of the 7 categories and for the remaining 57 scenes, we generated 2 tasks. This produced $57 * 7 * 2 + 2 * 7 * 1 = 812$ tasks

TABLE VI: **Quantitative comparison for Scene Generation.** We evaluate our against the baseline across diverse metrics measuring the visual realism (Real.), functionality (Func.), layout correctness (Lay.), Quality (Qual.), VQA score [17], and GPT Preference.

Method	VQA (↑)	Real. (↑)	Func. (↑)	Lay. (↑)	Compl. (↑)	Qual. (↑)	# Obj (↑)	GPT Pref. (↑)
Baseline	0.398 (±0.04)	6.889 (±0.36)	6.221 (±0.58)	6.166 (±0.42)	4.687 (±0.57)	5.991 (±0.24)	13.750 (±10.52)	18.000
Ours	**0.554** (±0.03)	**8.755** (±0.27)	**8.951** (±0.33)	**7.919** (±0.28)	**8.207** (±0.40)	**8.458** (±0.25)	**26.870** (±24.90)	**82.000**

TABLE VII: **Quantitative comparison across Difficulty Splits.** We evaluate our method against the baseline on Easy, Medium, and Hard splits. Our method consistently outperforms the baseline across all difficulty levels.

Method	VQA (↑)	Real. (↑)	Func. (↑)	Lay. (↑)	Compl. (↑)	Qual. (↑)	# Obj (↑)	GPT Pref. (↑)
Easy ($[0, 5]$ objects)								
Baseline	0.458 (±0.02)	6.767 (±0.36)	6.269 (±0.57)	6.079 (±0.48)	4.737 (±0.61)	5.963 (±0.26)	6.467 (±0.52)	33.333
Ours	**0.525** (±0.05)	**8.331** (±0.24)	**8.423** (±0.27)	**7.636** (±0.21)	**7.626** (±0.25)	**8.004** (±0.09)	**12.800** (±11.54)	**66.667**
Medium ($[6, 15]$ objects)								
Baseline	0.401 (±0.02)	6.933 (±0.37)	6.223 (±0.59)	6.199 (±0.41)	4.634 (±0.56)	5.997 (±0.22)	10.957 (±2.11)	17.143
Ours	**0.561** (±0.02)	**8.779** (±0.18)	**8.996** (±0.23)	**7.932** (±0.24)	**8.235** (±0.29)	**8.485** (±0.13)	**22.414** (±19.15)	**82.857**
Hard ($[16, 20]$ objects)								
Baseline	0.326 (±0.02)	6.808 (±0.30)	6.162 (±0.59)	6.099 (±0.42)	4.883 (±0.58)	5.988 (±0.30)	34.067 (±14.89)	6.667
Ours	**0.553** (±0.03)	**9.067** (±0.13)	**9.271** (±0.17)	**8.142** (±0.27)	**8.659** (±0.25)	**8.785** (±0.07)	**61.733** (±28.80)	**93.333**

the instruction and the programmatic success conditions across six dimensions: relation match (whether the spatial/logical relationship is preserved), target match (correctness of goal state), object match (whether referenced objects are correct), quantifier match (handling of "all," "any," or specific counts), instruction clarity (unambiguous and well-formed language), and physical feasibility (whether the task is achievable given typical robot capabilities). Each dimension was scored on a 0–1 scale, and we computed an aggregate alignment score as the weighted mean. The model additionally provided a categorical verdict—aligned, partially aligned, or misaligned—based on whether the termination conditions would correctly evaluate task success as described in the instruction.

Table IX shows that our method can successfully generate a variety of types of tasks appropriate to the assets in the scene. The **Alignment** score represents the overall instruction-code alignment, aggregating the six sub-dimensions. **Clarity** measures whether instructions are unambiguous and grammatically well-formed. **Feasibility** assesses physical realizability of the task. **Match** combines the four semantic dimensions (relation, target, object, quantifier) into a single score reflecting how accurately the code captures the instruction's intent. **Verdict** reports the percentage of tasks judged as fully aligned versus partially aligned (misaligned tasks, comprising approximately 1% of the dataset, are omitted for brevity). We additionally compute scene coverage metrics: object coverage measures the fraction of manipulable objects in each scene that appear in at least one generated task, while predicate coverage measures the fraction of available termination predicates used across tasks for that scene. All evaluations use temperature 0 for reproducibility, with automatic retry logic to handle rate limits.

The evaluation reveals strong overall task generation quality, with 0.91 mean alignment and 76% of tasks receiving full alignment verdicts. Performance varies by category: conjunction and recognition tasks achieve the highest alignment (0.97 and 0.96), likely because their success conditions map straightforwardly to compositional predicates, while color-based tasks show lower alignment (0.81), reflecting the challenge of grounding color references to specific object instances. High clarity (0.96) and semantic match (0.95) scores indicate that the generated instructions are well-formed and the termination conditions capture the intended semantics, though feasibility scores are slightly lower for spatial tasks (0.89) where precise placement requirements may exceed typical manipulation tolerances. The 88% object coverage demonstrates good utilization of scene assets, while the lower predicate coverage (29%) suggests the generator favors a subset of reliable predicates rather than exploring the full space of available success conditions—a conservative strategy that likely contributes to the high alignment scores. Overall, these results demonstrate our method can successfully generate a variety of types of tasks appropriate to the assets in the scene. While LLM-generated scenes and tasks are procedurally checked, we note that scenes and tasks require user review in order to ensure that they are aligned with the evaluation objective.

TABLE VIII: **Per-Scene Quantitative Analysis.** We report the performance breakdown across 10 distinct scene themes. Our method demonstrates robust generalization, outperforming the baseline in nearly all metrics across diverse environments.

Theme	Method	VQA (↑)	Real. (↑)	Func. (↑)	Lay. (↑)	Compl. (↑)	Qual. (↑)	# Obj (↑)	GPT Pref. (↑)
Bathroom Counter	Baseline	0.405 (±0.02)	6.901 (±0.37)	6.196 (±0.73)	6.260 (±0.26)	4.805 (±0.56)	6.041 (±0.19)	15.00 (±0.00)	10.00
	Ours	**0.564** (±0.02)	**8.913** (±0.16)	**9.159** (±0.22)	**7.967** (±0.28)	**8.276** (±0.33)	**8.579** (±0.15)	**28.50** (±18.47)	**90.00**
Classroom Supplies	Baseline	0.401 (±0.02)	6.881 (±0.31)	6.458 (±0.50)	5.931 (±0.41)	5.018 (±0.55)	6.072 (±0.17)	10.40 (±1.26)	**50.00**
	Ours	**0.562** (±0.02)	**8.790** (±0.18)	**8.990** (±0.18)	**7.842** (±0.28)	**8.406** (±0.26)	**8.507** (±0.12)	**32.90** (±23.70)	**50.00**
Craft Station	Baseline	0.399 (±0.02)	7.062 (±0.49)	6.385 (±0.56)	6.171 (±0.39)	4.590 (±0.60)	6.052 (±0.24)	9.70 (±0.48)	10.00
	Ours	**0.560** (±0.03)	**8.761** (±0.19)	**9.045** (±0.28)	**7.938** (±0.22)	**8.179** (±0.33)	**8.481** (±0.06)	**17.70** (±13.61)	**90.00**
Garage Workstation	Baseline	0.410 (±0.01)	7.032 (±0.34)	6.308 (±0.63)	6.246 (±0.47)	4.432 (±0.62)	6.005 (±0.26)	9.00 (±0.47)	0.00
	Ours	**0.566** (±0.02)	**8.796** (±0.21)	**9.004** (±0.20)	**7.881** (±0.28)	**8.207** (±0.29)	**8.472** (±0.13)	**13.00** (±11.68)	**100.00**
Garden Tools	Baseline	0.400 (±0.02)	7.018 (±0.45)	6.155 (±0.53)	6.315 (±0.47)	4.489 (±0.53)	5.994 (±0.21)	10.30 (±0.95)	30.00
	Ours	**0.561** (±0.02)	**8.778** (±0.16)	**8.949** (±0.15)	**7.950** (±0.20)	**8.175** (±0.27)	**8.463** (±0.12)	**13.80** (±11.36)	**70.00**
Kitchen Cabinet	Baseline	0.327 (±0.02)	6.862 (±0.31)	6.281 (±0.61)	6.008 (±0.39)	5.069 (±0.48)	6.055 (±0.26)	37.38 (±19.66)	12.50
	Ours	**0.554** (±0.03)	**9.052** (±0.13)	**9.313** (±0.17)	**8.070** (±0.25)	**8.710** (±0.25)	**8.786** (±0.06)	**48.88** (±25.63)	**87.50**
Laundry Sorting	Baseline	0.396 (±0.01)	6.795 (±0.24)	6.327 (±0.63)	6.269 (±0.34)	4.536 (±0.47)	5.982 (±0.21)	12.30 (±1.70)	0.00
	Ours	**0.558** (±0.03)	**8.742** (±0.17)	**8.975** (±0.26)	**8.021** (±0.16)	**8.202** (±0.28)	**8.485** (±0.12)	**35.30** (±27.34)	**100.00**
Office Desk	Baseline	0.457 (±0.02)	6.747 (±0.36)	6.183 (±0.25)	6.101 (±0.52)	4.697 (±0.59)	5.932 (±0.25)	7.00 (±0.00)	**57.14**
	Ours	**0.499** (±0.05)	**8.296** (±0.22)	**8.535** (±0.16)	**7.524** (±0.16)	**7.609** (±0.32)	**7.991** (±0.08)	**17.57** (±16.03)	42.86
Storage Room	Baseline	0.324 (±0.02)	6.747 (±0.29)	6.027 (±0.58)	6.203 (±0.45)	4.670 (±0.64)	5.912 (±0.35)	30.29 (±5.91)	0.00
	Ours	**0.552** (±0.02)	**9.085** (±0.14)	**9.224** (±0.17)	**8.224** (±0.28)	**8.601** (±0.24)	**8.783** (±0.08)	**76.43** (±26.40)	**100.00**
Tea Time	Baseline	0.458 (±0.02)	6.784 (±0.39)	6.345 (±0.77)	6.061 (±0.48)	4.773 (±0.67)	5.991 (±0.29)	6.00 (±0.00)	12.50
	Ours	**0.547** (±0.03)	**8.361** (±0.26)	**8.325** (±0.31)	**7.735** (±0.21)	**7.642** (±0.18)	**8.016** (±0.10)	**8.63** (±1.85)	**87.50**
Workshop Bench	Baseline	0.394 (±0.02)	6.838 (±0.35)	5.733 (±0.38)	6.199 (±0.47)	4.564 (±0.52)	5.833 (±0.22)	10.00 (±0.47)	20.00
	Ours	**0.556** (±0.03)	**8.675** (±0.10)	**8.846** (±0.22)	**7.928** (±0.26)	**8.200** (±0.30)	**8.412** (±0.16)	**15.70** (±10.36)	**80.00**

TABLE IX: LLM-judged quality metrics for 812 automatically generated manipulation tasks across 59 scenes and 7 competency axes.

		LLM Judge						**Coverage**	
Category	N	Alignment	Clarity	Feasibility	Match	Aligned%	Partial%	Object	Predicate
color	116	0.81	0.94	0.80	0.90	57	40	0.88	0.29
conjunction	116	0.97	0.98	1.00	0.98	91	9	0.88	0.29
counting	116	0.87	0.97	0.90	0.92	60	38	0.88	0.29
recognition	116	0.96	0.97	0.96	0.97	85	15	0.88	0.29
semantics	116	0.89	0.95	0.94	0.94	72	27	0.88	0.29
sorting	116	0.94	0.95	0.97	0.96	86	14	0.88	0.29
spatial	116	0.92	0.98	0.89	0.95	80	17	0.88	0.29
Overall	**812**	**0.91**	**0.96**	**0.92**	**0.95**	**76**	**23**	**0.88**	**0.29**

You are a scene generation expert creating REALISTIC robot manipulation scenarios.

REAL-WORLD SCENE PRINCIPLES:
1. Objects form CLUSTERS - not evenly spaced grids
2. Containers (bowls, bins) have objects INSIDE them
3. Supports (plates, trays) have objects ON TOP
4. Objects scatter naturally AROUND containers
5. Orientations VARY - not all aligned to 0°/90°

COORDINATE SYSTEM:
- Table bounds: X=[0.25 to 0.85], Y=[-0.40 to 0.40] (meters)
- Table center: (0.55, 0.0)
- Front=+X, Back=-X, Left=+Y, Right=-Y

PLACEMENT TYPES:
1. **place-on-base**: Object directly on table
{"type": "place-on-base", "object": "bowl_0", "x": 0.4, "y": 0.1, "yaw": 23}
VARY yaw angles (15, 47, 123, not just 0/90/180).
Position matters for anchors, less for loose objects.
2. **place-in**: Objects INSIDE a container
{"type": "place-in", "objects": ["apple_01", "orange_01"], "container": "bowl_0"}
Container MUST be placed first with place-on-base.
Great for fruits in bowls, items in bins.
3. **place-on**: Object ON TOP of support
{"type": "place-on", "object": "banana", "support": "plate_large", "position": "center"}
Support MUST be placed first.
position: "center", "edge", or "random"
Great for food on plates, items on trays.
4. **cluster-around**: Objects scattered NEAR an anchor
{"type": "cluster-around", "objects": ["mug", "spoon"], "anchor": "bowl_0", "radius": 0.15}
Creates natural groupings.

radius: how far from anchor (0.10–0.20m typical)

SCENE STRUCTURE (follow this pattern):
1. Place 1-2 ANCHOR objects (containers/supports) on table
2. Put objects INSIDE containers (place-in)
3. Put objects ON supports (place-on)
4. Cluster objects AROUND anchors (cluster-around)
5. Add a few LOOSE objects to fill space

REALISTIC SPACING:
- Anchors: 0.25-0.35m apart
- Clustered objects: 0.08-0.15m from anchor
- Loose objects: fill remaining space naturally

Fig. 14: System prompt for Stage I (Semantic Planning). This prompt instructs the LLM to generate physically plausible scene layouts using structured predicates rather than raw coordinates.

OUTPUT FORMAT (JSON only, no markdown):

```
{
"objects": [
{"name": "bowl_0"},
{"name": "plate_large"},
{"name": "apple_01"},
{"name": "orange_01"},
{"name": "banana"},
{"name": "mug"},
{"name": "spoon"}
],
"predicates": [
{"type": "place-on-base", "object": "bowl_0", "x": 0.40, "y": 0.15, "yaw": 23},
{"type": "place-on-base", "object": "plate_large", "x": 0.65, "y": -0.10, "yaw": 156},
{"type": "place-in", "objects": ["apple_01", "orange_01"], "container": "bowl_0"},
{"type": "place-on", "object": "banana", "support": "plate_large", "position": "center"},
{"type": "cluster-around", "objects": ["mug", "spoon"], "anchor": "bowl_0", "radius": 0.12}
]
}
```

CRITICAL RULES:

1. Object names MUST match EXACTLY from catalog
2. Containers/supports MUST be placed before objects go in/on them
3. Create INTERESTING scenes with containment, stacking, AND clustering
4. VARY yaw angles - real scenes aren't grid-aligned
5. Return ONLY valid JSON, no markdown

Fig. 15: Continued System prompt for Stage I (Semantic Planning). This prompt instructs the LLM to generate physically plausible scene layouts using structured predicates rather than raw coordinates.

SCENE REQUEST: theme from dataset
TARGET: target object count objects

TABLE SIZE: 0.7m $\times$ 1.0m = 0.70m^2 (objects must fit with spacing!)

SIZE LIMITS (max 1-2 large objects, prefer smaller for 8+ items):
Large ($> 0.08\text{m}^2$): computed from catalog footprint
Avoid picking multiple large objects - they won't all fit!

AVAILABLE OBJECTS:
CONTAINERS (can hold objects inside): filled from catalog

SUPPORTS (can stack objects on): filled from catalog

FOOD: filled from catalog

TOOLS: filled from catalog

OTHER: filled from catalog

MEDIUM SCENE STRATEGY (10-14 objects):
- Use 1-2 containers/supports as ANCHORS
- Put 2-4 objects IN containers (place-in)
- Stack 1-2 items ON supports (place-on)
- Cluster 2-3 objects near anchors (cluster-around)
- VARY yaw angles - no grid alignment!

SUGGESTED for diversity (use only if they fit the theme): preselected objects

Fig. 16: User prompt template for Stage I (medium target count). The highlighted fields are populated at runtime (theme, target count, size warnings, catalog subsets, and diversity suggestions). Analogous strategy blocks are used for sparse (fewer than 10) and dense (15+) targets.

PREVIOUS ATTEMPT FAILED:
feedback string produced by spatial/physical solver or grammar checks

Please fix the issues. Common fixes:
- Use MORE containment (place-in) to reduce table crowding
- Use MORE stacking (place-on) to utilize vertical space
- Use clustering (cluster-around) instead of individual placements
- Select SMALLER objects if collisions persist

Fig. 17: Feedback block appended to the user prompt when spatial solving, physical placement, grammar checks, or intersection validation fails. The highlighted region is the dynamic diagnostic message.

Algorithm 2 Physical Placement Solver

Input: Objects B, Predicates P, Solved Base Poses
Output: 3D coordinates (x, y, z) for all objects

{Solve Stacking:}
for all $p \in P$ where p.type $==$ place-on **do**
  $s \leftarrow p$.support
  $B_{\text{peers}} \leftarrow \{b' \mid b'$ is already on $s\}$
  $(x, y) \leftarrow$ FindSpot$(s, p.\text{object}, B_{\text{peers}})$
  $p.\text{object}.z \leftarrow s.z + s.\text{height} + p.\text{object.height}/2$
  $p.\text{object}.(x, y) \leftarrow (x, y)$
end for
{Solve Containment:}
for all $p \in P$ where p.type $==$ place-in **do**
  $c \leftarrow p$.container
  if TotalArea$(p.\text{objects}) > 0.8 \times$ Area(c) **then**
    $p.\text{objects} \leftarrow$ SortAndFilter$(p.\text{objects}, c.\text{capacity})$
  end if
  $(R, C) \leftarrow$ ComputeGridDimensions$(c.\text{dims}, |p.\text{objects}|)$
  for $i = 0$ **to** $|p.\text{objects}| - 1$ **do**
    $(r, c) \leftarrow (i//C, i\%C)$
    $(x_{\text{loc}}, y_{\text{loc}}) \leftarrow$ GridCellCenter$(r, c, c.\text{dims})$
    Jitter$(x_{\text{loc}}, y_{\text{loc}})$
    $p.\text{objects}[i].(x, y) \leftarrow c.(x, y) + (x_{\text{loc}}, y_{\text{loc}})$
    $p.\text{objects}[i].z \leftarrow c.z + c.\text{height}/2 + \text{buffer}$
  end for
end for
return Success